

Nearest Greater Element to the Right

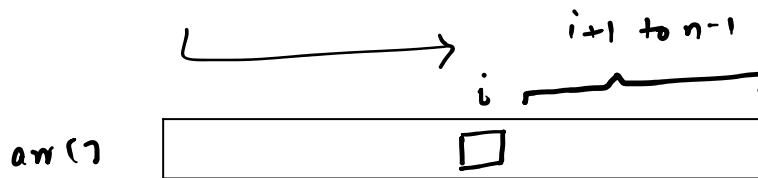
Given an array of **N** integers, design an algorithm to find, for each element, the **nearest greater element** to its **right**.

Note : for an element without a greater element to its right, output -1.

Example :

Input	5	3	6	7	2	4	1
Output	6	6	7	-1	4	4	-1

↑ ↑ ↑ ↑ ↑ ↑ ↑

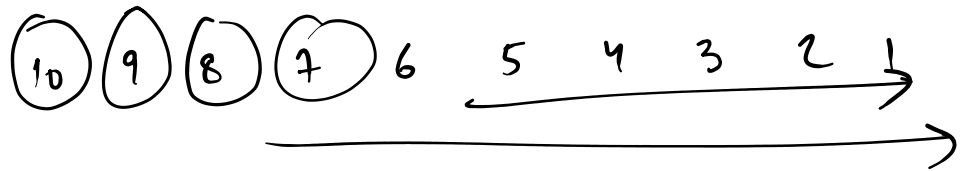


find the ngr of $arr[i]$?

```

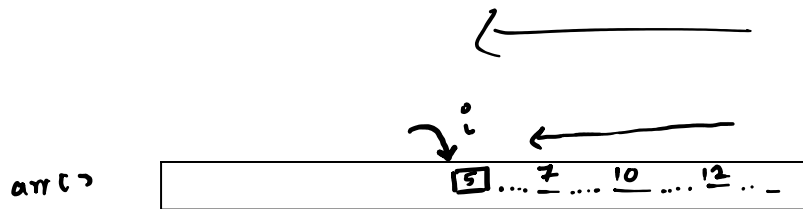
for (i = 0; i < n; i++) {
    // find the ngr of arr[i]
    for (j = i + 1; j < n; j++) {
        if (arr[j] > arr[i]) {
            // arr[j] is the ngr of arr[i]
            break;
        }
    }
}

```

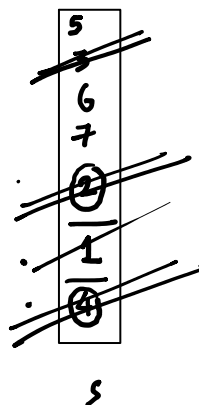
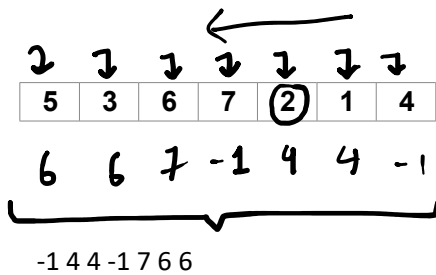


$arr[0] : 1 \text{ to } n-1$ $n-1$
 $arr[1] : 2 \text{ to } n-1$ $n-2$
 $arr[2] : 3 \text{ to } n-1$ $n-3$
 $\vdots$
 $\vdots$
 $arr[n-2] : n-1 \text{ to } n-1$ 1
 $arr[n-1] :$ 0

$\left. \begin{array}{l} n-1 \\ n-2 \\ n-3 \\ \vdots \\ 1 \\ 0 \end{array} \right\} \sum \sim n^2$



find the ngr of $arr[i]$ in const time?



(

①	0	2	3	4
2	2	3	4	-1

1
0
2
3
4

Nearest Greater Element to the Left

Nearest Greater Element to the Left

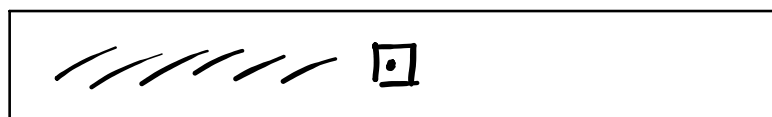
Given an array of **N** integers, design an algorithm to find, for each element, the **nearest greater element** to its **left**.

Note : for an element without a greater element to its left, output **-1**.

Example :

Input	5	2	0	4	1	3	6
Output	-1	5	2	5	4	4	-1

“ “ “ “ “ “ “
-1 5 2 5 4 4 -1



find the ngl of arr[i]
in const time ?

Stock Span Problem

Given prices of a stock for **N** consecutive days, design an algorithm to find, for each day, **span of stock price** on that day.

We define **span of stock price today** as the **maximum** number of consecutive days (starting from today and going backwards) for which the stock price was **less than or equal** to today's price.

Example :

	0	1	2	3	4	5	6
Input	100	80	60	70	60	75	85
Output	1	1	1	2	1	4	6

	0	1	2	3	4	5	6	7
Input	100	80	60	70	60	75	85	200
Output	1	1	1	2	1	4	6	8

$7 - (-1) = 8$

	0	1	2	3	4	5	6	7
Input	100	80	60	70	60	75	85	200
Output	1	1	1	2	1	4	6	8

	0	1	2	3	4	5	6	7
Input	100	80	60	70	60	75	85	200
	$j = -1$	$j = 0$	$j = 1$	$j = 1$	$j = 3$	$j = 1$	$j = 0$	$j = -1$
	1	1	1	2	1	4	6	8

- ~~(200, 7)~~
- ~~(85, 6)~~
- ~~(75, 5)~~
- ~~(60, 4)~~

(75, 5)
(60, 4)
(70, 3)
(60, 2)
(80, 1)
(100, 0)

1 1 1 2 1 4 6 8

132-Pattern

Given an array of **N** integers **nums**, design an algorithm to check if there exists a **132-Pattern** in **nums**. We define a **132-Pattern** as a subsequence of 3-integers **nums[i]**, **nums[j]** and **nums[k]** such that

- $i < j < k$ and
- $nums[i] < nums[k] < nums[j]$.

Example :

N = 6

	0	1	2	3	4	5
Input	3	4	6	2	1	5

Output : true

Nearest Smaller Element to the Right

Given an array of **N** integers, design an algorithm to find, for each element, the **index** of **nearest smaller element** to its **right**.

Note : for an element without a smaller element to its right, output **N**

Example :

	0	1	2	3	4	5	6
Input	2	5	1	3	6	4	0
Output	2	2	6	6	5	6	7

Nearest Smaller Element to the Left

Given an array of **N** integers, design an algorithm to find, for each element, the **index** of **nearest smaller element** to its **left**.

Note : for an element without a smaller element to its left, output **-1**

Example :

	0	1	2	3	4	5	6
Input	0	3	5	4	1	6	2
Output	-1	0	1	1	0	4	4

Number of Valid Subarrays

Given an integer array **nums**, design an algorithm to find the number of non-empty sub-arrays with the leftmost element of the subarray not larger than other elements in the subarray.

Example :

	0	1	2	3	4	5
Input	1	4	2	5	3	9

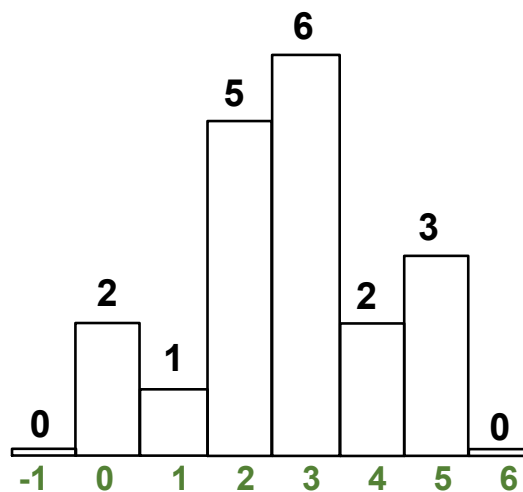
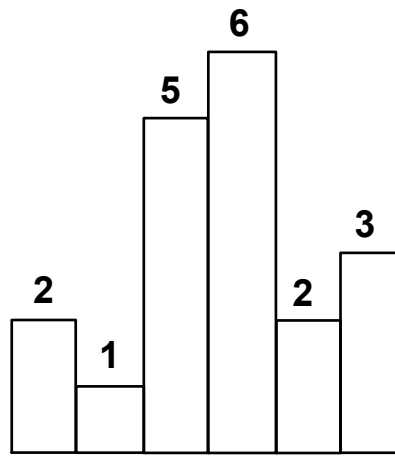
Output : 15

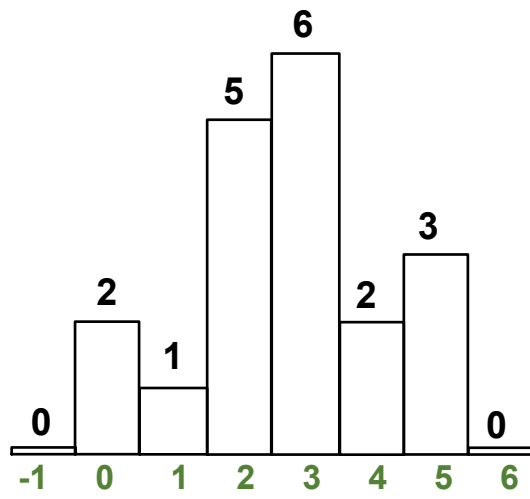
Largest Rectangle in Histogram

Given an array of **N** integers representing **heights** of bars in a histogram where width of each bar is a **1 unit**, design an algorithm to find the **area of the largest rectangle** in the histogram.

Example :

0	1	2	3	4	5
2	1	5	6	2	3





[HW] Maximal Rectangle

<https://leetcode.com/problems/maximal-rectangle/>

Nearest Greater Element to the Right

Nearest Greater Element to the Right

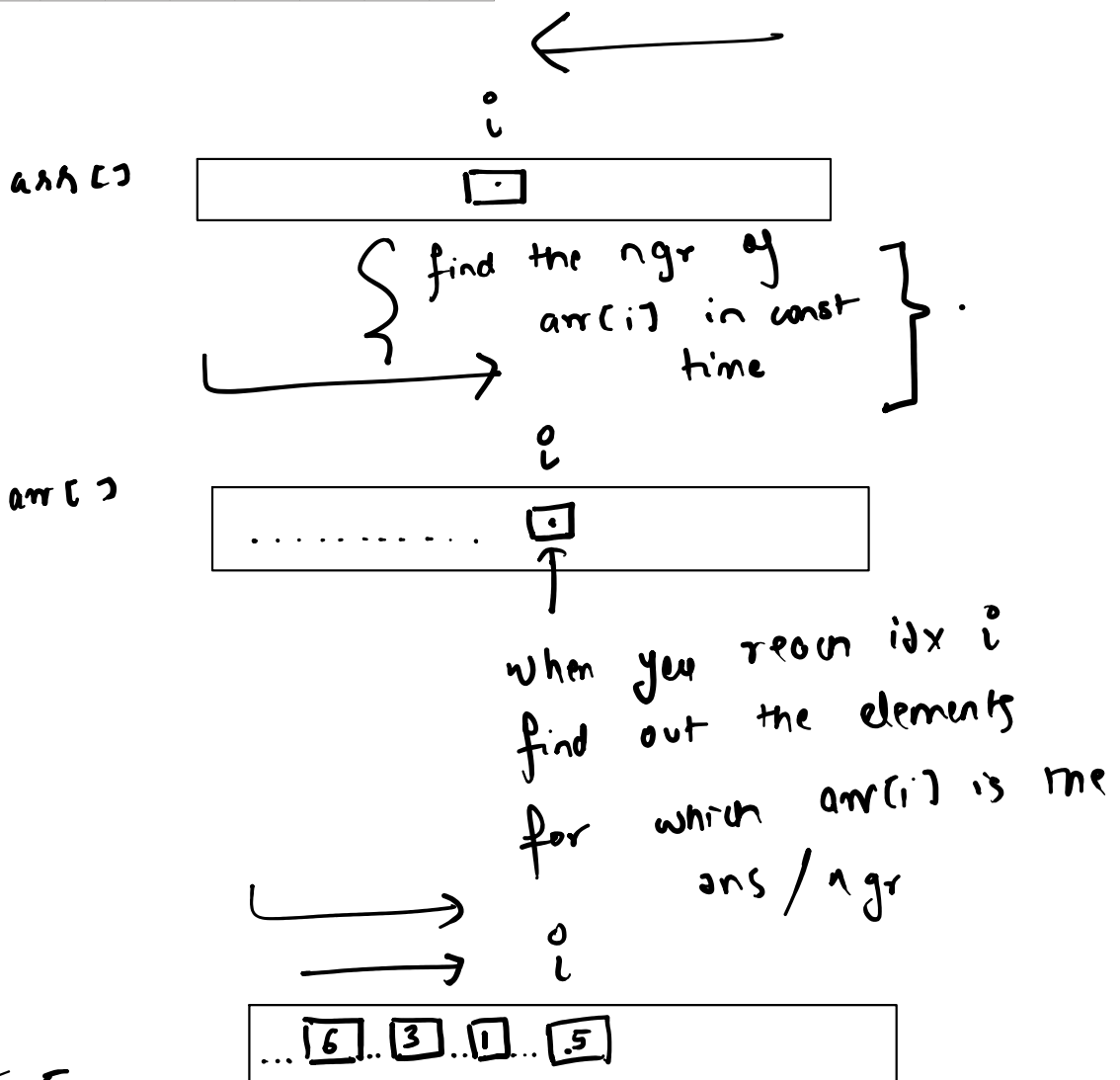
You must scan the array
L to R and solve

Given an array of **N** integers, design an algorithm to find, for each element, the **nearest greater element** to its **right**. $O(n)$ time

Note : for an element without a greater element to its right, output -1.

Example :

Input	5	3	6	7	2	1	4
Output	6	6	7	-1	4	4	-1



5
1
3
6
5
5

stk is used to

↳ STK is
 used to
 track el- for which
 we are yet to compute the
 ans

(4, 6)

(1, 5)
(2, 4)
(7, 3)
(6, 2)
(3, 1)
(5, 0)

	0	1	2	3	4	5	6
Input	5	3	6	7	2	1	4
				↓			↓
	0	1	2	3	4	5	6
ans	6	6	7	-1	4	4	-1

s → to track elements for which
 we are yet to find ans

[HW] Next Greater Element IV

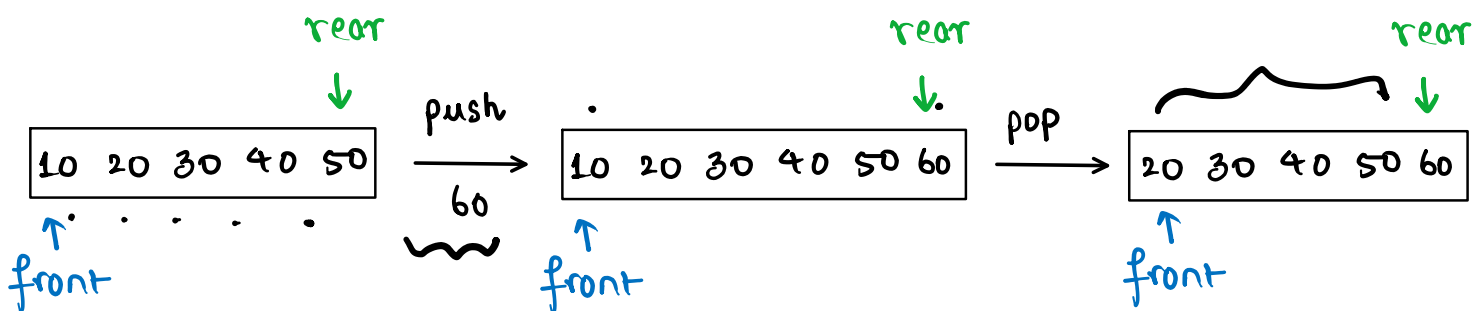
<https://leetcode.com/problems/next-greater-element-iv/>

[HW] Next Greater Node in Linked List

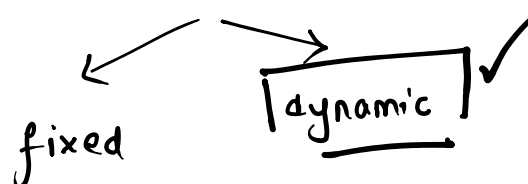
<http://leetcode.com/problems/next-greater-node-in-linked-list/>

Queue

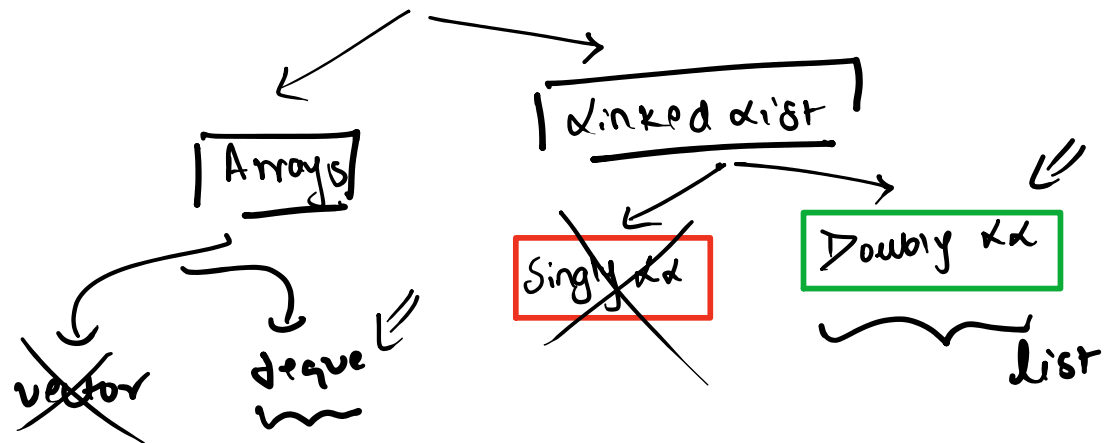
- It is a linear data-structure that supports insertion from one-end (rear) and deletion from the other-end (front).
- Queue operates according to the FIFO principle.



How to implement Queue ?



How to represent Queue?



Queue Operations

- 1. push (data)
- 2. pop()
- 3. front()
- 4. size() }
- 5. empty()

back()

→ Dynamic Queue using Deque

q ~~10 | 20 | ...~~

Logical View

d 10 | 20

Internal View.

q.push(10)
q.push(20)
⋮
q.front()
q.pop()
q.size()
q.empty()

d.pb(10)
d.pb(20)
⋮
d[0] / d.front()
d.pop-front()
d.size()
d.empty()

First Non-Repeating Character in a Stream

First Non-Repeating Character in a Stream

Given a stream of characters, design an algorithm to find, for each character, **first non-repeating character** in the stream.

Note : output -1 if upon reading a character there are no non-repeating characters in the stream.

Example :

	0	1	2	3	4	5	6	
Input	a	a	b	c	d	b	c	$a \rightarrow \infty$ $b \rightarrow \infty$ $c \rightarrow \infty$ $d \rightarrow 1$
Output	a	-1	b	b	b	c	d...	

	0	1	2	3	4	5	6	
Input	a	a	b	c	d	b	c	$a \rightarrow \infty$ $b \rightarrow \infty$ $c \rightarrow \infty$ $d \rightarrow 1$
	a	-1	b	b	b	c	d...	

a ~~a~~ ~~b~~ ~~c~~ ~~d~~
to track non-repeating char of the stream we've seen so far

a b c c a b a
= a a a a b -1 -1

~~a~~ ~~b~~ ~~c~~

$a \rightarrow \infty$
 $b \rightarrow \infty$
 $c \rightarrow \infty$

[HW] Implement a Stack using Two Queues

Single Queue

Implement a Stack Class using Two Queues

Impl. a Queue using Two / Single Stack